Python File: btt006_unit_7_assignment-solution.py

```
# Module 5: Build an LLM OCR App (Note for facilitators)
# -
# - Students complete TWO tasks by filling in specific lines of code:
#
# Task 1 - Multimodal content structure:
#   - Students see: "content": ...
#   - Students write: Complete list with text part and image_url part
#   - This tests: Understanding how to structure multimodal messages (text + image)
#   - SOLUTION PROVIDED: [{"type": "text", "text": prompt}, {"type": "image_url", "image_url": {"url": f"data:i
#
# Tasks 2 - Structured output format:
#   - Students see: response_format=...
#   - Students write: {"type": "json_object"}
#   - This tests: Understanding how to request structured JSON output from the API
#   - SOLUTION PROVIDED: {"type": "json_object"}
#
# -----
#
# This app uses OpenAI's multimodal capabilities (GPT-4o) to extract
# structured data from images and PDFs.
# Complete the code below to:
# 1. Send both text AND images to the API (multimodal messages)
# 2. Request structured JSON output for reliable data extraction


# Instructions:
# 1. Read through the guide
# 2. Find the BEGIN SOLUTION / END SOLUTION blocks below and complete it. DO NOT remove
#    the BEGIN SOLUTION / END SOLUTION comments
# 3. Run 'streamlit run app.py' in the terminal
# 4. Test your chatbot in the browser

import streamlit as st
from openai import OpenAI
import base64
import io
from PIL import Image
import json
import fitz  # PyMuPDF for PDF processing

# Initialize OpenAI client
client = OpenAI()

# ============================================
# PAGE CONFIGURATION
# ============================================

st.set_page_config(
    page_title="eCornell OCR App",
    page_icon="■",
    layout="wide"
)

# ============================================
# CORNELL HEADER
# ============================================

col1, col2 = st.columns([1, 4])
with col1:
    st.image(
        "cornell_seal.png",
        width=100,
    )
with col2:
    st.markdown(
        "<h3 style='color: #b31b1b; margin-bottom: 0;'>■ Invoice & Receipt Data Extractor</h3>",
        unsafe_allow_html=True,
    )
    st.caption("Powered by eCornell")

st.markdown("---")

st.markdown(
    "Upload a PDF, PNG, JPG, or JPEG file containing an invoice or receipt. "
    "The app will extract structured data using OpenAI's multimodal model."
```

```
)

# ============================================
# HELPER FUNCTIONS
# ============================================

def image_to_base64(image: Image.Image, format="PNG") -> str:
    """
    Converts a PIL Image to a base64 encoded string.
    This format is required for sending images to the OpenAI API.
    """
    buffered = io.BytesIO()
    # Ensure image is in RGB format for consistency
    image.convert("RGB").save(buffered, format=format)
    img_byte = buffered.getvalue()
    return base64.b64encode(img_byte).decode('utf-8')

@st.cache_data(show_spinner="Processing document...")
def pdf_to_images(file_bytes: bytes, dpi=150) -> list[Image.Image]:
    """
    Converts PDF bytes to a list of PIL Images using PyMuPDF.
    Each page becomes a separate image that can be analyzed.
    Cached to avoid reprocessing the same PDF on reruns.
    """
    images = []
    try:
        doc = fitz.open(stream=file_bytes, filetype="pdf")
        for page_num in range(len(doc)):
            page = doc.load_page(page_num)
            # Increase resolution for better OCR quality
            zoom = dpi / 72  # 72 is the default DPI in PDF
            mat = fitz.Matrix(zoom, zoom)
            pix = page.get_pixmap(matrix=mat)
            img = Image.frombytes("RGB", [pix.width, pix.height], pix.samples)
            images.append(img)
        doc.close()
    except Exception as e:
        st.error(f"Error converting PDF to images: {e}")
    return images

@st.cache_data(show_spinner="Extracting data from image...")
def process_image_with_api(image_base64: str, prompt: str) -> str:
    """
    Wrapper function to make API calls cacheable.
    Caches results based on image content and prompt to avoid redundant API calls.
    """
    return get_structured_data_from_image(image_base64, prompt)

# ============================================
# MULTIMODAL API CALL WITH STRUCTURED OUTPUT (SOLUTION)
# ============================================

def get_structured_data_from_image(image_base64: str, prompt: str):
    """
    Sends image (base64) and prompt to OpenAI multimodal model and requests structured JSON output.

    This function demonstrates:
    1. How to structure multimodal messages (text + image)
    2. How to request structured JSON output using response_format
    """
    try:
        response = client.chat.completions.create(
            model="gpt-4o",
            messages=[
                {
                    "role": "system",
                    "content": "You are an expert assistant specialized in extracting structured information fr
                },
                {
                    "role": "user",
                    # BEGIN SOLUTION
                    "content": [{"type": "text", "text": prompt}, {"type": "image_url", "image_url": {"url": f"
                    # END SOLUTION
                }
            ],
            # BEGIN SOLUTION
            response_format={"type": "json_object"},
            # END SOLUTION
            max_tokens=1000
        )
```

```
        return response.choices[0].message.content
    except Exception as e:
        st.error(f"Error calling OpenAI API: {e}")
        return None

# ============================================
# DEFINE EXTRACTION PROMPT
# ============================================

# This prompt tells the AI exactly what information to extract and how to format it
json_prompt = """
Please extract the following information from the document image provided and structure it as a JSON object:

- `vendor_name`: The name of the seller or vendor.
- `transaction_date`: The date of the transaction (use format YYYY-MM-DD).
- `items`: A list of items purchased. Each item should be an object with:
    - `description`: Description of the item or service.
    - `price`: Total price for that item.
- `tax_amount`: The total amount of tax charged (if available).
- `total_amount`: The final total amount paid.
- `payment_method`: How the bill was paid (e.g., 'Credit Card', 'Cash', or last 4 digits if available).

If a field is not found or not applicable, use `null` or omit the field. Ensure the output is only the JSON obj
"""

# ============================================
# FILE UPLOADER
# ============================================

st.markdown("---")

uploaded_file = st.file_uploader(
    "Upload your document",
    type=["pdf", "png", "jpg", "jpeg"],
    help="Supported formats: pdf, png, jpg, jpeg"
)

# ============================================
# MAIN PROCESSING LOGIC
# ============================================

if uploaded_file is not None:
    st.info(f"■ File uploaded: **{uploaded_file.name}** (Type: {uploaded_file.type})")

    processing_placeholder = st.empty()

    images_to_process = []
    file_bytes = uploaded_file.getvalue()

    # Convert uploaded file to image(s)
    with st.spinner("Processing file... Converting to image(s)..."):
        if uploaded_file.type == "application/pdf":
            processing_placeholder.info("Detected PDF. Converting pages to images...")
            images_to_process = pdf_to_images(file_bytes)

        elif uploaded_file.type.startswith("image/"):
            processing_placeholder.info("Detected Image. Loading...")
            try:
                img = Image.open(io.BytesIO(file_bytes))
                images_to_process = [img]  # Treat as a list of one image
            except Exception as e:
                st.error(f"Error reading image file: {e}")
                st.stop()
        else:
            st.error(f"Unsupported file type: {uploaded_file.type}")
            st.stop()

    if not images_to_process:
        processing_placeholder.error("Could not extract any images from the uploaded file.")
        st.stop()
    else:
        processing_placeholder.success(
            f"Successfully processed file. Found {len(images_to_process)} image(s) to analyze."
        )

        st.markdown("---")
        st.subheader("Document Preview & Extracted Data")

        # Display images and extraction results side-by-side
        col1, col2 = st.columns(2)
```

```
with col1:
    st.markdown("**Document Image(s):**")
    for i, img in enumerate(images_to_process):
        st.image(
            img,
            caption=f"Page {i+1}" if len(images_to_process) > 1 else "Document Image",
            use_container_width=True
        )

with col2:
    st.markdown("**Extracted JSON Data:**")
    all_json_results = []

    for i, img in enumerate(images_to_process):
        page_num = i + 1
        st.markdown(
            f"*Analyzing {'page ' + str(page_num) if len(images_to_process) > 1 else 'image'}...*"
        )

        with st.spinner(
            f"Sending {'page ' + str(page_num) if len(images_to_process) > 1 else 'image'} to OpenAI..."
        ):
            # Convert image to base64 format
            img_base64 = image_to_base64(img)

            # Call the cached wrapper to avoid redundant API calls on rerun
            api_response = process_image_with_api(img_base64, json_prompt)

        if api_response:
            # Try to extract JSON from the response
            try:
                # Find JSON within markdown code fences or raw JSON
                json_start = api_response.find("```json")
                json_end = api_response.rfind("```")

                if json_start != -1 and json_end != -1:
                    json_string = api_response[json_start + 7:json_end].strip()
                elif api_response.strip().startswith('{'):
                    json_string = api_response.strip()
                else:
                    st.warning(
                        f"Could not find JSON block in response for page {page_num}. "
                        f"Raw response:\n```\n{api_response}\n```"
                    )
                    json_string = None

                if json_string:
                    parsed_json = json.loads(json_string)
                    st.json(parsed_json)
                    all_json_results.append(parsed_json)
                else:
                    all_json_results.append({
                        "error": f"No valid JSON found in response for page {page_num}"
                    })

            except json.JSONDecodeError as json_err:
                st.error(f"Error parsing JSON response for page {page_num}: {json_err}")
                st.text("Raw Response:")
                st.code(api_response, language=None)
                all_json_results.append({
                    "error": f"Failed to parse JSON for page {page_num}",
                    "raw_response": api_response
                })
            except Exception as e:
                st.error(
                    f"An unexpected error occurred while processing the response for page {page_num}: {
                )
                all_json_results.append({
                    "error": f"Unexpected error processing response for page {page_num}"
                })
        else:
            st.error(f"Failed to get response from OpenAI API for page {page_num}.")
            all_json_results.append({"error": f"API call failed for page {page_num}"})

        # Add separator between pages (if multi-page document)
        if i < len(images_to_process) - 1:
            st.markdown("---")

    # Download button for all extracted data
if all_json_results:
```

```
            st.download_button(
                label="Download All Extracted Data as JSON",
                data=json.dumps(
                    all_json_results if len(all_json_results) > 1 else all_json_results[0],
                    indent=2
                    ,
                file_name=f"{uploaded_file.name}_extracted_data.json",
                mime="application/json"
            )

else:
    st.info("■■ Upload a document to begin.")

# ============================================
# FOOTER
# ============================================

st.markdown("---")

st.markdown(
    "<div style='text-align: center; color: gray;'>"
    "© eCornell<br>"
    "For assistance, contact course staff"
    "</div>",
    unsafe_allow_html=True,
)
```